

7/11/2022.

AJA Color Box Remote Demos:

The Color Box from AJA supports a rich RESTful API using OpenAPI. C++, Go and Python API implementations are included in this package. To demonstrate some of the functionality available via the RESTful API some simple demo projects have been provided as well.

This document gives an overview of the API and describes those demo projects and sample scripts.

OpenAPI

The Color Box uses the OpenAPI specification to generate our RESTful API. The demos mostly focus on the Qt with C++ implementation but the Python and Go implementation is included as well. All the demos use an OpenAPI generated api.

After unzipping the package:

Full C++ Implementation location.

- `demos/cpp-qt/api`

Full Python Implementation location.

- `demos/python/api`

Full Go Implementation location

- `demos/go/api`

The OpenAPI interface to Colorbox is self-documenting and that can be found at:

- `ipaddressOfColorBox/api`(for example `http://192.168.1.140/api`).

This allows you not only to see the data structures for the API but also try it out.

C++ Demo Projects

There are 6 Qt C++ demos included. These are built with Qt 5.15.2(6.2.4 on Mac M1) but earlier versions should work as well. Windows, Mac and Linux are supported. On Mac M1 Qt 6.2.4 is required.

Each demo project requires the user to enter the IP Address of the Color Box Converter. To do so just type in the IP Address and press return. Before you enter the address, it will look like this:

IPAddress NOT CONNECTED

After it will show it is connected and the IP Address:

IPAddress CONNECTED

The C++ Qt api(demos/cpp-qt/api) is used for these demo projects. This API uses Qt signals and slots. For every call into the API the client will get a signal on success and different signal if there is an error. For example, as a response to the `getFrameStore` API call these connects should be made.

```
// API related slots
connect(&_amp;, &OAIDefaultApi::getFrameStoreSignal, this, &Dialog::handleGetFrameStore);
connect(&_amp;, &OAIDefaultApi::getFrameStoreSignalE, this, &Dialog::handleGetFrameStoreError);
```

On success, `handleGetFrameStore` will be called and on error, `handleGetFrameStoreError` will get called.

If you are just setting a parameter you don't need to have a signal/slot setup but it is recommended in shipping code so any error can be logged. Here is an example of how to put the ColorBox pipeline in "LUT" mode once the box is connected.

```
// Now put the ColorBox in LUT mode. Note: if it was in Orion Mode it will be ~2 seconds
before LUT mode is ready as it needs to change firmware.
OAITransformMode transformMode;
transformMode.setValue(OAITransformMode::eOAITransformMode::LUT);
OAISystemConfig systemConfig;
systemConfig.setTransformMode(transformMode);
_ampi.setSystemConfig(systemConfig);
```

The valid choices for transform mode are(except of course `INVALID_VALUE_OPENAPI_GENERATED`) :

```
enum class eOAITransformMode {
    INVALID_VALUE_OPENAPI_GENERATED = 0,
    LUT,
    COLORFRONT,
    ORION,
    BBC,
    NBC
};
```

Building C++ Demo Projects

The preferred method of building the C++ demo projects is cmake with Qt Creator. In addition there is a demos.pro file that can be used with qmake with Qt Creator.

To build with cmake you need a cmake enabled Qt Creator kit(for M1 Mac you need Qt 6.2.4). For all other platform 5.15.2 should be fine.

1. Unzip demos(on windows make sure path is not too long as windows has a path limit of 250).
2. Get github opensource ntv2 sdk(<https://github.com/aja-video/ntv2/archive/refs/tags/v16.2-bugfix5.zip>) unzip and put it in demos/cpp-qt/ntv2sdk so that ajalibraries is right below ntv2sdk
3. Open Qt Creator and open CMakeLists.txt in cpp-qt dir.
4. Right-click on "Project" and run cmake.

Right-click on "Project" and run Build

NOTE: Step 2 is optional and only used by FrameGrab and PreviewDemo to for ancillary data packet detection and decoding.

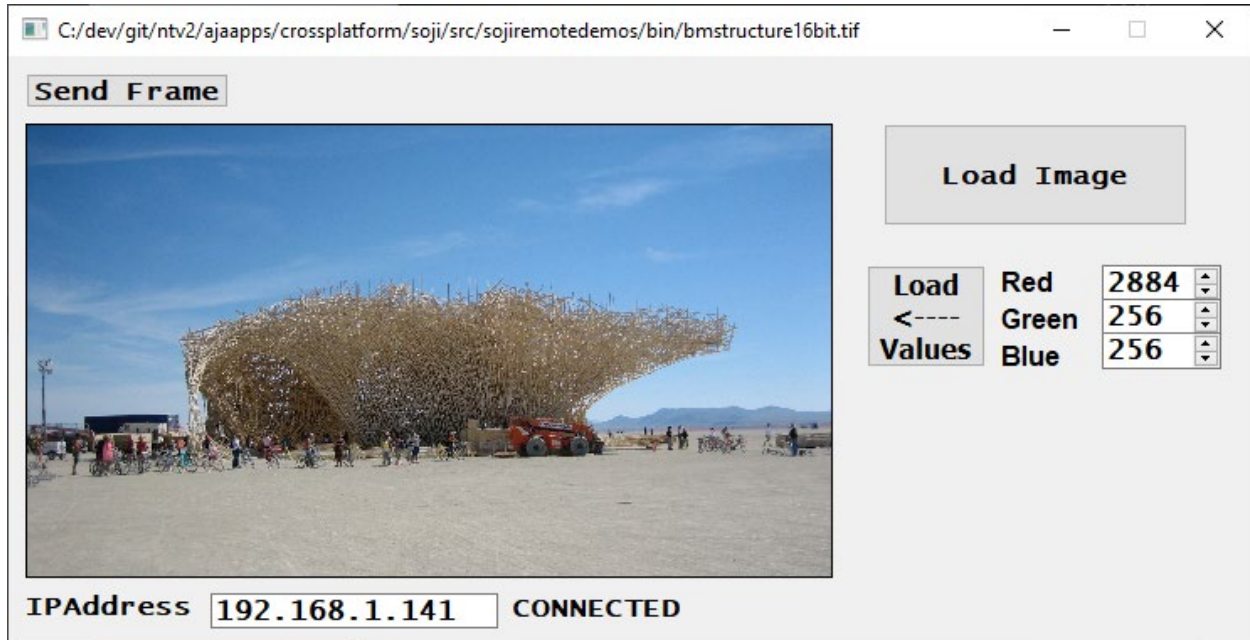
Linux demos will need to have libtiff installed.

MacOS 10 demos needs libtiff and pkg-config via homebrew.

Windows demos come with libtiff in package.

SojiDynamicFrameLoad

This demo shows how to transfer an image to Soji over WebSockets and the RESTful API. The RESTful API allows the Color Box to go into Dynamic “Store” mode and set a Video Format. The WebSocket is then used to transfer the image.

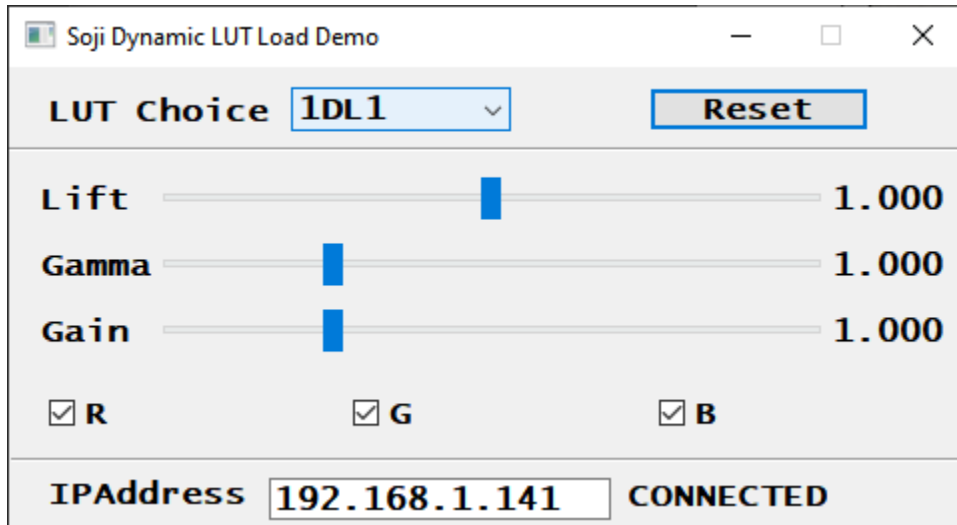


In the demo, the image can come from a flat matte by loading RGB Values or from a 16 Bit Tiff. But in general the image just needs to be a 16 bit RGB image in memory that is transferred via WebSockets to the Color Box. Once that ColorBox is Connected simply press the “Send Frame” button and the Color Box will go into Dynamic Frame Store mode and the image will be updated. For this demo the Color Box format is set to 1080p23.98 but it can be set to “AUTO” to follow the input format. For the WebSocket transfer to work the raster size of the image needs to match the current video format raster size on the Color Box.

In a future version, this can be used to upload and overlay(.png) to the ColorBox.

SojiDynamicLutLoad

This demo shows how to dynamically load any of the 1D LUTs or 3D LUT on the Color Box Pipeline via WebSockets and the RESTful API. The RESTful API is used to make the “LUT Choice” and to set that LUT into Dynamic transfer mode. The WebSocket is then used to transfer either a 1DLUT or 3DLut. In this case the 3DLUT is derived from the 1DLUT for demo purposes.



Soji Dynamic LUT Load Demo

LUT Choice **1DL1** **Reset**

Lift **1.000**

Gamma **1.000**

Gain **1.000**

☒ **R** ☒ **G** ☒ **B**

IPAddress **192.168.1.141** **CONNECTED**

The ColorBox pipeline supports many different pipelines and for this above example it requires the LUT pipeline to loaded. This can be done by the following code:

```
// Now put the ColorBox in LUT mode. Note: if it was in Orion Mode it will be ~2 seconds
before LUT mode is ready as it needs to change firmware.
OAITransformMode transformMode;
transformMode.setValue(OAITransformMode::eOAITransformMode::LUT);
OAISystemConfig systemConfig;
systemConfig.setTransformMode(transformMode);
_api.setSystemConfig(systemConfig);
```

For Colorspace converter setup see:

```
void Dialog::dynmicLutChoiceChanged(index)
```

SojiDynamicMatrixLoad

This demo shows how to dynamically load any of the 3x4 Matrices on the Color Box Pipeline via WebSockets and the RESTful API. The RESTful API is used to make the “Matrix Choice” and to set that LUT into Dynamic transfer mode. The WebSocket is then used to transfer the matrix coefficients. For ease QMatrix4x4 is used in this demo.

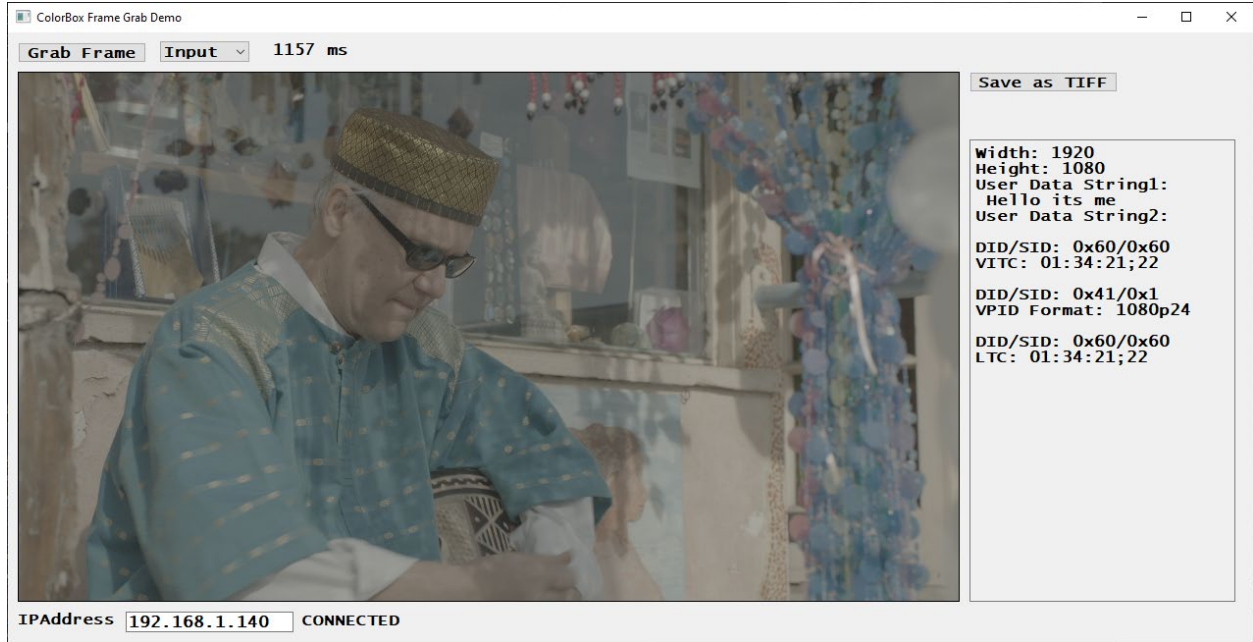
The screenshot shows a window titled "Soji Dynamic Matrix Load Demo". It features a "Matrix Choice" dropdown menu set to "MTX2" and a "Reset" button. Below this are four sliders for "ProcAmp Gain", "ProcAmp Black", "ProcAmp Hue", and "ProcAmp Saturation". The sliders are positioned at 1.000, 0.000, 0.000, and 1.000 respectively. At the bottom, there is an "IPAddress" field containing "192.168.1.141" and a "CONNECTED" status indicator.

Parameter	Value
Matrix Choice	MTX2
ProcAmp Gain	1.000
ProcAmp Black	0.000
ProcAmp Hue	0.000
ProcAmp Saturation	1.000
IPAddress	192.168.1.141
Status	CONNECTED

SojiFrameGrab

The SojiFrameGrab Demo shows how to use WebSocket messaging to get an uncompressed full size frame grab from the Color Box. The demo can also be setup to get the associated ancillary metadata for that frame and decode it using the NTV2 API.

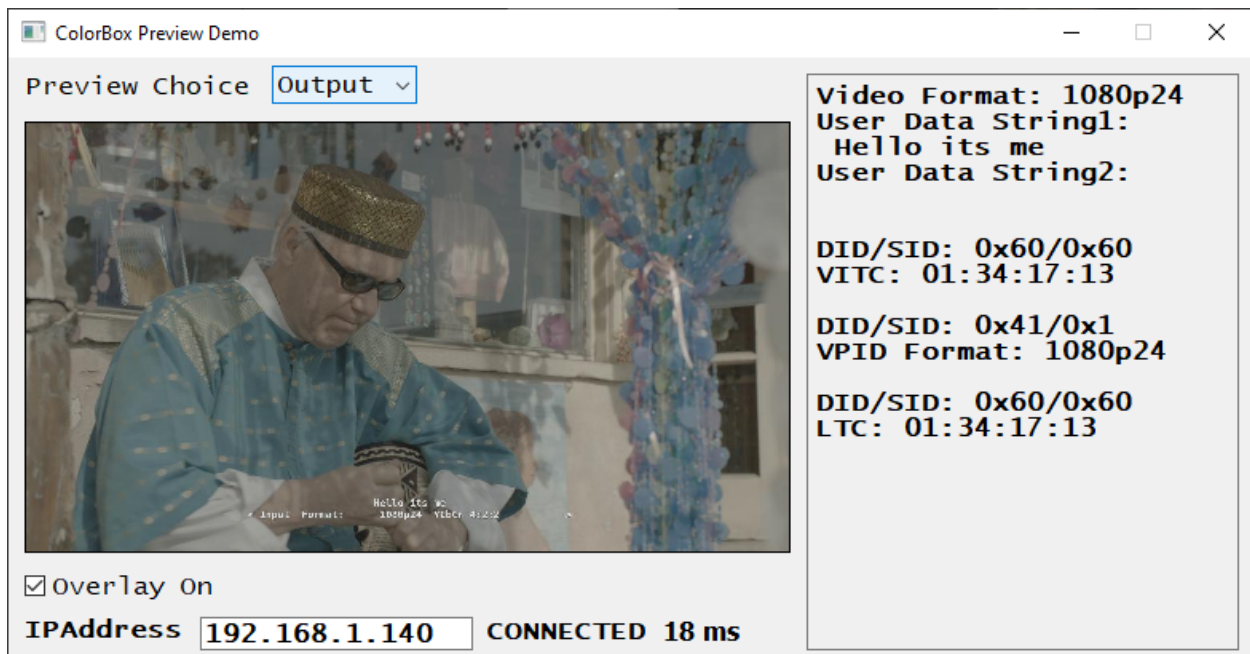
The demo allows the capture of the unprocessed (but color space converted) Input or capture the processed Output (if overlay is on then the overlay is also transferred).



SojiPreviewDemo

The SojiPreviewDemo uses the RESTful API to get a proxy jpg preview of the Color Box. The demo can also be setup to get the associated ancillary metadata for that frame and decode it using the NTV2 API.

In addition, the demo demonstrates how to use the PreviewTap...The PreviewTap can be set to either Input or Output. If it is set to Input then the retrieved preview is before an Color Box processing has been done. If it is set to Output, then the pipeline processing has been done before the preview image is sent back. In addition, if the Overlay checkbox is set the overlay will be applied to the preview image as well.



SojiRGBTriplet

This demo shows how to download a 16 bit RGB Triplet to the ColorBox to a particular pixel location and then read back that pixel location having applied the current transforms. For instance, if Color Box is in LUT mode the API will place the RGB triplet in the frame store and the ColorBox Software will calculate the expected results of that RGB triplet running through the pipeline

1DLUT -> 3x3 Matrix -> 3DLUT -> 1DLUT -> 3x3 Matrix-> 1DLUT

and report back both the expected and actual results. For this to work the Pipeline needs to be in "Store" mode and not being updated with Input Video. This will also work with the Orion and Colorfront engine.

SojiRGBTriplet

X

600

Y

400

R

1234

G

2345

B

3456

Cookie

Test 3

Request Data

366 ms

Results

2022-03-28T14:31:41: for cookie 'Test 1' with coords(350,450) sent rgb(1000,2000,3000), expect rgb(1000,2000,3000) and got rgb(1000,2000,3000)
2022-03-28T14:32:11: for cookie 'Test 2' with coords(800,200) sent rgb(1234,2345,3456), expect rgb(1234,2345,3456) and got rgb(1234,2345,3456)
2022-03-28T14:33:19: for cookie 'Test 3' with coords(600,400) sent rgb(1234,2345,3456), expect rgb(1655,1837,2183) and got rgb(1655,1838,2183)

IPAddress

192.168.1.141

CONNECTED

[Python Demos](#)

Please read `demos/python/ README.md` for details.

[Go Demos](#)

Please read `demos/go/ README.md` for details.

7/19/2022 – Updated FrameGrab demo so it can control to capture the unprocessed Input vs. the processed Output.

7/11/2022 – Updated PreviewDemo so it can control PreviewTap(Input vs. Output) and Overlay On/Off.